

<question>You apply k-NN to a dataset with features: age (years) and salary (USD). Salary ranges from 1000 to 100000, age from 18 to 60. What will most likely happen without preprocessing?

<variant>Model will ignore salary completely

<variant>Model will give equal importance to both features

<variant>Model will fail to run

<variant>Model will automatically normalize features

<variantright>Salary will dominate distance calculations and bias predictions

<question>You train a k-NN classifier with k=1 and observe 100% accuracy on training data but poor test performance. What is the most likely issue?

<variant>Underfitting

<variant>Data leakage

<variant>High bias

<variant>Poor feature scaling

<variantright>Overfitting to noise in training data

<question>You split your dataset incorrectly and include test data in training. What happens?

<variant>Model underfits

<variant>Model becomes slower

<variant>Model ignores features

<variant>Model cannot train

<variantright>Evaluation becomes overly optimistic due to data leakage

<question>In the code below, what is missing before training k-NN?

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
knn.fit(X_train, y_train)
```

<variant>Model deployment

<variant>Confusion matrix

<variant>Label shuffling

<variant>Removing all numeric columns

<variantright>Feature scaling such as StandardScaler

<question>What does this code create? `knn = KNeighborsClassifier(n_neighbors=5)`

<variant>A regression model with 5 outputs

<variant>A clustering model with 5 clusters

<variant>A neural network with 5 layers

<variant>A model that removes 5 features

<variantright>A k-NN classifier using 5 nearest neighbors

<question>What is the main problem in this code?

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y)
```

<variant>The scaler should not be used with k-NN

<variant>The labels should be scaled too

<variant>The model should be trained before splitting

<variant>The test size is missing

<variantright>Data leakage because scaling is done before train-test split

<question>What is the likely issue with this code for k-NN?

```
X = df[['age', 'income', 'city']]
```

```
knn.fit(X_train, y_train)
```

<variant>Age cannot be used in ML

<variant>Income must always be removed

<variant>Only one feature is allowed

<variant>Target variable is missing from X

<variantright>City is categorical and must be encoded before using k-NN

<question>What does this pipeline help prevent?

```
pipe = Pipeline([ ('scaler', StandardScaler()), ('knn', KNeighborsClassifier(n_neighbors=5)) ]),
```

<variant>Overwriting the dataset

<variant>Removing all outliers automatically

<variant>Creating fake labels

<variant>Changing classification into regression

<variantright>Incorrect preprocessing and data leakage during model training

<question>What is the main goal of regression?

<variant>Predict categories

<variant>Group similar samples

<variant>Reduce dataset size

<variant>Detect anomalies

<variantright>Predict numeric values

<question>Which of the following is a regression task?

<variant>Classifying emails

<variant>Recognizing digits

<variant>Clustering users

<variant>Detecting fraud types

<variantright>Predicting house prices

<question>What does a linear regression model assume?

<variant>No structure in data

<variant>Output is categorical

<variant>Features are unrelated

<variant>No relation exists

<variantright>Linear relationship exists

<question>What is k-fold cross-validation?

<variant>Use k features

<variant>Train k models

<variant>Use k datasets

<variant>Increase data size

<variantright>Rotate k splits

<question>Why is cross-validation useful?

<variant>Speeds up training

- <variant>Removes features
- <variant>Guarantees accuracy
- <variant>Shrinks dataset
- <variantright>Gives stable estimate

<question>What is a drawback of cross-validation?

- <variant>Not for regression
- <variant>Removes labels
- <variant>Changes dataset
- <variant>Reduces accuracy
- <variantright>Higher computation cost

<question>You add many irrelevant features to a regression model. What is the likely effect?

- <variant>Model becomes faster
- <variant>Error always decreases
- <variant>Predictions stay same
- <variant>Data becomes cleaner
- <variantright>Model may overfit noise

<question>You compare two models using cross-validation. Model A has stable scores, Model B has highly varying scores. What does this suggest?

- <variant>Model A is slower
- <variant>Model B uses fewer features
- <variant>Model A overfits
- <variant>Model B is simpler
- <variantright>Model B is unstable

<question>You use k-fold cross-validation with $k=2$. What is a drawback?

- <variant>Too many splits
- <variant>High computation
- <variant>No training data
- <variant>Too much shuffling
- <variantright>High variance estimate

<question>You use cross-validation to choose between models. What is the main goal?

- <variant>Reduce dataset size
- <variant>Remove noise
- <variant>Speed up training
- <variant>Change features
- <variantright>Select best model

<question>You fit regression with highly correlated features and notice large coefficient values. Why?

- <variant>Model too simple
- <variant>Data too small
- <variant>Labels missing
- <variant>No scaling
- <variantright>Multicollinearity issue

<question>You apply linear regression without intercept on shifted data. What happens?

- <variant>No predictions
- <variant>Perfect fit

<variant>Data disappears
<variant>No training
<variantright>Biased predictions

<question>What does this code measure?`mean_squared_error(y_test, y_pred)`
<variant>Classification accuracy
<variant>Number of folds
<variant>Model complexity
<variant>Feature correlation
<variantright>Average squared error

<question>What does this pipeline help prevent?`pipe = Pipeline([ ('scaler', StandardScaler()), ('model', LinearRegression())])cross_val_score(pipe, X, y, cv=5)`
<variant>Low training speed
<variant>Missing target values
<variant>Too many folds
<variant>Negative predictions
<variantright>Preprocessing leakage

<question>What does this code display?`pd.DataFrame({ 'feature': X.columns, 'coef': model.coef_})`
<variant>Train-test split
<variant>Model predictions
<variant>Validation folds
<variant>Residual values
<variantright>Feature coefficients

<question>Which method combines LASSO and Ridge?
<variant>PCA
<variant>KNN
<variant>Decision tree
<variant>Linear regression
<variantright>Elastic Net

<question>What does this code do?`Ridge(alpha=1.0)`
<variant>Removes features
<variant>Clusters data
<variant>Scales dataset
<variant>Splits data
<variantright>Applies L2 penalty

<question>What does this code do?`Lasso(alpha=0.1)`
<variant>Applies L2 penalty
<variant>Performs clustering
<variant>Scales features
<variant>Splits dataset
<variantright>Applies L1 penalty

<question>Why is regularization useful in high-dimensional data?
<variant>Reduces dataset size
<variant>Speeds up prediction
<variant>Removes labels

<variant>Changes model type
<variantright>Prevents overfitting

<question>What does λ (lambda) control in regularization?

<variant>Number of features
<variant>Dataset size
<variant>Model type
<variant>Learning rate
<variantright>Strength of penalty

<question>What happens when λ is very large?

<variant>Model becomes complex
<variant>Error becomes zero
<variant>Features increase
<variant>Training speeds up
<variantright>Coefficients shrink strongly

<question>What happens when $\lambda = 0$?

<variant>No training occurs
<variant>Features are removed
<variant>Loss becomes zero
<variant>Model fails
<variantright>Becomes ordinary regression

<question>What does accuracy measure?

<variant>Only correct positives
<variant>Only correct negatives
<variant>Error magnitude
<variant>Prediction probability
<variantright>All correct predictions ratio

<question>Given TP=50, TN=40, FP=10, FN=0, what is accuracy?

<variant>0.80
<variant>0.85
<variant>0.88
<variant>0.95
<variantright>0.90

<question>What is precision?

<variant>TP divided by all actual positives
<variant>TN divided by all negatives
<variant>Correct predictions over total
<variant>FN divided by predictions
<variantright>TP divided by predicted positives

<question>What function is used in logistic regression?

<variant>Linear function
<variant>Exponential only
<variant>Square function

<variant>Step function
<variantright>Sigmoid function

<question>What is the sigmoid function output range?

<variant>-1 to 1
<variant> $-\infty$ to ∞
<variant>0 to ∞
<variant> $-\infty$ to 0
<variantright>0 to 1

<question>What is the decision threshold in logistic regression?

<variant>0.0
<variant>1.0
<variant>-1.0
<variant>Depends on features
<variantright>0.5 by default

<question>What is AUC?

<variant>Average accuracy
<variant>Feature importance
<variant>Loss value
<variant>Threshold value
<variantright>Area under ROC

<question>AUC = 0.5 means?

<variant>Perfect model
<variant>Overfitting
<variant>Underfitting
<variant>High precision
<variantright>Random performance

<question>AUC close to 1 means?

<variant>Poor model
<variant>Random model
<variant>Unstable model
<variant>Low recall
<variantright>Strong classifier

<question>What happens if FP increases?

<variant>Recall increases
<variant>Accuracy increases
<variant>TN increases
<variant>FN decreases
<variantright>Precision decreases

<question>What happens if FN increases?

<variant>Precision increases
<variant>TN decreases
<variant>Accuracy increases
<variant>FP decreases
<variantright>Recall decreases

<question>Given confusion matrix $\begin{bmatrix} 50 & 10 \\ 5 & 35 \end{bmatrix}$, what is accuracy?

<variant>0.80

<variant>0.82

<variant>0.83

<variant>0.84

<variantright>0.85

<question>You run logistic regression and get this warning:ConvergenceWarning: lbfgs failed to convergeWhat is the most likely fix?

<variant>Reduce dataset size

<variant>Remove target variable

<variant>Shuffle labels

<variant>Use predict instead of fit

<variantright>Increase max_iter or scale features

<question>What is the issue in this code?model = LogisticRegression()model.fit(X_train, y_train)print(model.coef_)Features are on very different scales

<variant>No issue, coefficients are always comparable

<variant>Logistic regression ignores feature scale

<variant>coef_ returns probabilities

<variant>Scaling only affects speed

<variantright>Coefficients are not comparable without scaling

<question>You run logistic regression on perfectly separable data. What happens?

<variant>Model cannot train

<variant>Predictions are random

<variant>Loss becomes constant

<variant>No coefficients learned

<variantright>Coefficients may grow very large

<question>What is the issue here?model = LogisticRegression(penalty='l1')model.fit(X_train, y_train)

<variant>L1 cannot be used with classification

<variant>fit requires probabilities

<variant>y_train must be continuous

<variant>Penalty is ignored

<variantright>Solver must support L1 penalty

<question>You run this code:model = LogisticRegression()model.fit(X_train, y_train)y_pred = (model.predict_proba(X_test)[:,-1] > 0.3)What is the main effect?

<variant>Precision increases

<variant>Model becomes faster

<variant>Loss decreases

<variant>Accuracy fixed

<variantright>Recall increases

<question>What happens in this code?tree =

DecisionTreeClassifier(random_state=42)tree.fit(X_train, y_train)tree2 =

DecisionTreeClassifier(random_state=42)tree2.fit(X_train, y_train)

<variant>Different splits always

<variant>Models cannot match

<variant>Second model fails

<variant>Random state ignored
<variantright>Reproducible same model

<question>Why set random_state in training and splitting?

<variant>Increase accuracy
<variant>Scale features
<variant>Reduce overfitting
<variant>Remove noise
<variantright>Make results repeatable

<question>What is the issue here?tree.fit(X_train, y_train)print(tree.score(X_train, y_train))Score is 1.00.

<variant>Model failed
<variant>Data has no patterns
<variant>Tree is underfitting
<variant>Score method is broken
<variantright>Possible overfitting

<question>What is the better next check after training score 1.00?

<variant>Increase max_depth
<variant>Remove test data
<variant>Use training score only
<variant>Disable random_state
<variantright>Evaluate validation score

<question>What is the issue in this code?grid = GridSearchCV(tree, param_grid, cv=5)grid.fit(X, y)print(grid.best_score_)

<variant>GridSearchCV cannot fit
<variant>Tree cannot use full data
<variant>cv needs test data
<variant>Labels are ignored
<variantright>No separate holdout test

<question>What does this code most likely cause?tree =

DecisionTreeClassifier(max_depth=None, min_samples_leaf=1)tree.fit(X_train, y_train)

<variant>Strong underfitting
<variant>No tree growth
<variant>Random predictions
<variant>Feature scaling error
<variantright>High overfitting risk

<question>What is the hidden issue in this code?scores = cross_val_score(tree, X_test, y_test, cv=5)

<variant>cross_val_score needs probabilities
<variant>Decision trees cannot use CV
<variant>cv must be 1
<variant>X_test must be scaled
<variantright>Validation is done on holdout data

<question>What split is preferred in decision trees?

<variant>Highest entropy
<variant>Random split

- <variant>Smallest dataset
- <variant>Largest feature
- <variantright>Max information gain

<question>A node has entropy 1. After split, weighted entropy is 0.4. What is information gain?

- <variant>0.2
- <variant>0.4
- <variant>0.5
- <variant>0.8
- <variantright>0.6

<question>If a split does not change entropy, what happens?

- <variant>Model improves
- <variant>Training faster
- <variant>Features increase
- <variant>Data shrinks
- <variantright>No useful split

<question>What happens if all splits give low information gain?

- <variant>Tree grows faster
- <variant>Features removed
- <variant>Model overfits
- <variant>Data changes
- <variantright>Tree stops splitting

<question>What is the effect of noisy data on entropy-based splits?

- <variant>Entropy decreases
- <variant>Tree becomes shallow
- <variant>Features removed
- <variant>No splits occur
- <variantright>Tree may overfit noise

<question>What happens if max_depth is not limited in entropy-based trees?

- <variant>Underfitting
- <variant>No splits
- <variant>Less variance
- <variant>Faster model
- <variantright>Overfitting

<question>What happens if tree grows without limits?

- <variant>Underfitting
- <variant>No splits
- <variant>Less variance
- <variant>Faster model
- <variantright>Overfitting

<question>What is the purpose of train_test_split?

- <variant>Scale data
- <variant>Remove noise
- <variant>Add features
- <variant>Shuffle labels
- <variantright>Split dataset

<question>What is a good practice before splitting data?

<variant>Train model

<variant>Evaluate model

<variant>Remove labels

<variant>Predict values

<variantright>Shuffle data

<question>What is the issue if data is not shuffled?

<variant>Model faster

<variant>Better accuracy

<variant>No effect

<variant>Less variance

<variantright>Biased split

<question>What happens if we tune hyperparameters on test data?

<variant>Better model

<variant>Faster training

<variant>Lower variance

<variant>More features

<variantright>Biased results

<question>What is a hyperparameter?

<variant>Learned weight

<variant>Prediction value

<variant>Error metric

<variant>Feature value

<variantright>Predefined model setting

<question>Which is a hyperparameter in decision trees?

<variant>Prediction label

<variant>Loss value

<variant>Accuracy score

<variant>Residual

<variantright>Max depth

<question>Why do we keep a holdout test set separate from validation?

<variant>To increase training data

<variant>To speed up training

<variant>To remove noise

<variant>To balance classes

<variantright>To get unbiased evaluation

<question>What is the main role of the validation set?

<variant>Train the model

<variant>Store predictions

<variant>Normalize features

<variant>Remove outliers

<variantright>Select hyperparameters

<question>Why is data preprocessing important before modeling?

<variant>To increase dataset size

<variant>To remove labels

- <variant>To reduce features
- <variant>To change model type
- <variantright>To improve data quality

<question>What is the main issue with categorical features in ML models?

- <variant>They are too large
- <variant>They reduce accuracy
- <variant>They increase speed
- <variant>They remove patterns
- <variantright>They are not numeric

<question>What does one-hot encoding do?

- <variant>Scales features
- <variant>Removes categories
- <variant>Sorts values
- <variant>Combines features
- <variantright>Creates binary columns

<question>What is a drawback of one-hot encoding?

- <variant>Removes information
- <variant>Reduces accuracy
- <variant>Slows training always
- <variant>Changes labels
- <variantright>Increases feature count

<question>What happens if polynomial degree is too low?

- <variant>Overfitting
- <variant>Noise learning
- <variant>High variance
- <variant>Perfect fit
- <variantright>Underfitting

<question>What does this code do?PolynomialFeatures(degree=2)

- <variant>Removes features
- <variant>Scales data
- <variant>Splits dataset
- <variant>Encodes categories
- <variantright>Creates interaction terms

<question>What is the output of polynomial features degree=2 for one feature x?

- <variant>x only
- <variant> x^2 only
- <variant>1 only
- <variant> x^3 only
- <variantright>1, x, x^2

<question>What is the correct approach to avoid leakage with polynomial features?

- <variant>Apply after prediction
- <variant>Use only test data
- <variant>Remove polynomial features
- <variant>Shuffle dataset
- <variantright>Use pipeline with split

<question>What is the main problem with missing data?

- <variant>Increases dataset size
- <variant>Speeds up training
- <variant>Improves accuracy
- <variant>Reduces features
- <variantright>Breaks model assumptions

<question>Which is a simple method to handle missing values?

- <variant>Duplicate data
- <variant>Shuffle labels
- <variant>Remove target
- <variant>Scale features
- <variantright>Fill with mean

<question>When is dropping rows with missing values risky?

- <variant>Dataset is large
- <variant>Features are numeric
- <variant>Data is balanced
- <variant>Model is simple
- <variantright>Too much data removed

<question>What is mean imputation best for?

- <variant>Categorical data
- <variant>Text features
- <variant>Binary labels
- <variant>Time series
- <variantright>Numeric features

<question>What is a drawback of undersampling?

- <variant>More data
- <variant>Better accuracy
- <variant>Faster model
- <variant>No noise
- <variantright>Loss of information

<question>What is a drawback of oversampling?

- <variant>Less data
- <variant>Faster training
- <variant>No effect
- <variant>Lower variance
- <variantright>Overfitting risk

<question>What does SMOTE do?

- <variant>Remove features
- <variant>Shuffle dataset
- <variant>Scale values
- <variant>Split data
- <variantright>Generate synthetic samples

<question>What is the issue in this code?

```
X_filled = df.fillna(df.mean())
```

```
X_train, X_test = train_test_split(X_filled)
```

<variant>Mean cannot be used

<variant>fillna is invalid

<variant>train_test_split needs y

<variant>Scaling missing

<variantright>Imputation before split

<question>What is the issue here? Dataset is highly imbalanced

```
model.fit(X_train, y_train)y_pred = model.predict(X_test)accuracy_score(y_test, y_pred)
```

<variant>Model not trained

<variant>Prediction invalid

<variant>X_test missing

<variant>y_test wrong

<variantright>Accuracy is misleading

<question>What is the correct fix for imbalanced evaluation?

<variant>Increase dataset size

<variant>Remove labels

<variant>Shuffle data

<variant>Use regression

<variantright>Use F1 or recall

<question>What does this code do?

```
LogisticRegression(class_weight='balanced')
```

<variant>Removes imbalance

<variant>Scales features

<variant>Splits data

<variant>Encodes labels

<variantright>Weights minority class

<question>What is ensemble learning?

<variant>Using one strong model

<variant>Removing weak models

<variant>Scaling data

<variant>Reducing features

<variantright>Combining multiple models

<question>What is the main goal of ensemble methods?

<variant>Reduce dataset size

<variant>Increase noise

<variant>Simplify features

<variant>Speed up training

<variantright>Improve prediction performance

<question>What is bagging?

<variant>Sequential training

<variant>Feature scaling

<variant>Removing samples

<variant>Encoding data

<variantright>Parallel training on subsets

<question>What does each tree in Random Forest use?

- <variant>Full dataset always
- <variant>Same features
- <variant>No labels
- <variant>Scaled data only
- <variantright>Random subset of data

<question>What is feature randomness in Random Forest?

- <variant>Removing features
- <variant>Scaling features
- <variant>Sorting features
- <variant>Encoding features
- <variantright>Random feature selection

<question>What is AdaBoost?

- <variant>Bagging method
- <variant>Clustering algorithm
- <variant>Scaling technique
- <variant>Regression only method
- <variantright>Boosting algorithm

<question>What does AdaBoost focus on during training?

- <variant>Correct samples
- <variant>Random samples
- <variant>Features only
- <variant>Noise removal
- <variantright>Misclassified samples

<question>What happens to weights in AdaBoost?

- <variant>Stay constant
- <variant>Random change
- <variant>Removed
- <variant>Scaled equally
- <variantright>Misclassified increase

<question>What is Gradient Boosting?

- <variant>Parallel training
- <variant>Random sampling
- <variant>Feature scaling
- <variant>Clustering method
- <variantright>Sequential residual learning

<question>What does Gradient Boosting optimize?

- <variant>Accuracy only
- <variant>Dataset size
- <variant>Feature count
- <variant>Training speed
- <variantright>Loss function

<question>What is the main risk of boosting methods?

- <variant>Underfitting
- <variant>No learning

- <variant>Data loss
- <variant>Feature removal
- <variantright>Overfitting

<question>What happens if boosting runs too many iterations?

- <variant>Model simplifies
- <variant>No predictions
- <variant>Data shrinks
- <variant>Features removed
- <variantright>Overfitting risk

<question>What is learning rate in boosting?

- <variant>Dataset size
- <variant>Number of features
- <variant>Model type
- <variant>Split size
- <variantright>Step size for updates

<question>What happens if learning rate is very small?

- <variant>Faster training
- <variant>Overfitting immediately
- <variant>No learning
- <variant>Less data used
- <variantright>Needs more iterations

<question>What happens if learning rate is large?

- <variant>More stable model
- <variant>Slower training
- <variant>Better accuracy
- <variant>Less variance
- <variantright>Unstable learning

<question>What is the issue in this code?`model = RandomForestClassifier(n_estimators=1)`

- <variant>Too many trees
- <variant>No labels used
- <variant>Cannot train
- <variant>Scaling missing
- <variantright>No ensemble effect

<question>What does this code do?`RandomForestClassifier(n_estimators=100)`

- <variant>Single tree
- <variant>Sequential trees
- <variant>Scaling data
- <variant>Removing features
- <variantright>Builds many trees

<question>What is unsupervised learning?

- <variant>Learning with labels
- <variant>Predicting targets
- <variant>Using regression
- <variant>Classification method
- <variantright>Learning without labels

<question>What is the goal of clustering?

<variant>Predict values

<variant>Classify data

<variant>Reduce features

<variant>Scale data

<variantright>Group similar points

<question>Which algorithm is used for clustering?

<variant>Linear regression

<variant>Logistic regression

<variant>Decision tree

<variant>Random forest

<variantright>KMeans

<question>Why is scaling important in KMeans?

<variant>Remove clusters

<variant>Reduce features

<variant>Speed training

<variant>Encode labels

<variantright>Balance distances

<question>What is the issue in this code?kmeans =

KMeans(n_clusters=3)kmeans.fit(X_scaled)kmeans.fit(X_test)

<variant>Scaling missing

<variant>KMeans cannot fit

<variant>X_test invalid

<variant>Clusters too many

<variantright>Test data used again

<question>You run KMeans with very large K (close to number of samples). What happens?

<variant>Better generalization

<variant>Faster training

<variant>Clusters merge

<variant>Less variance

<variantright>Each point becomes cluster

<question>What is the issue in this code?kmeans = KMeans(n_clusters=3)kmeans.fit(X)centers = kmeans.cluster_centers_kmeans.fit(X_new)

<variant>cluster_centers_ invalid

<variant>KMeans cannot refit

<variant>X_new must be labeled

<variant>Too many clusters

<variantright>Previous clusters overwritten

<question>You apply KMeans on dataset with many irrelevant features. What happens?

<variant>Better clustering

<variant>Faster model

<variant>No effect

<variant>Clusters become smaller

<variantright>Distances become noisy

<question>You use KMeans for customer segmentation, but clusters change significantly after adding new data. Why?

<variant>KMeans ignores new data

<variant>Clusters are fixed

<variant>Scaling removed

<variant>Features lost

<variantright>Centroids depend on data distribution

<question>You run this pipeline on customer data: `pipe = Pipeline([('scaler', StandardScaler()), ('kmeans', KMeans(n_clusters=4))])` `pipe.fit(X_train)` `labels = pipe.predict(X_test)` Clusters on test data look completely different from training. What is the most likely cause?

<variant>Pipeline cannot predict

<variant>KMeans cannot generalize

<variant>Too many clusters

<variant>Scaling removed clusters

<variantright>Train and test distributions differ

<question>You run KMeans and then evaluate clusters like this: `labels = kmeans.fit_predict(X_scaled)` `score = silhouette_score(X_scaled, labels)` You get a high score, but clusters are useless in business. Why?

<variant>Silhouette cannot be computed

<variant>KMeans is supervised

<variant>Scaling incorrect

<variant>Too many samples

<variantright>Metric ignores business meaning

<question>You cluster data and then add a new feature: `kmeans.fit(X_old)` `labels_old = kmeans.labels_` `kmeans.fit(X_new)` `labels_new = kmeans.labels_` Labels change drastically. Why?

<variant>KMeans lost data

<variant>Labels are random

<variant>Clusters cannot update

<variant>Scaling missing

<variantright>Feature space changed distances

<question>You run KMeans with this code: `kmeans = KMeans(n_clusters=3, random_state=42)` `labels = kmeans.fit_predict(X_scaled)` You notice one cluster has only 2 points while others have thousands. What does this suggest?

<variant>Model is perfect

<variant>Data is balanced

<variant>K is too small

<variant>Scaling is wrong

<variantright>Outliers form cluster

<question>You want to use KMeans for anomaly detection: `kmeans.fit(X_scaled)` `distances = kmeans.transform(X_scaled)` Which idea is correct?

<variant>Small distances are anomalies

<variant>Clusters define anomalies directly

<variant>Labels show anomalies

<variant>Centroids are anomalies

<variantright>Large distances indicate anomalies

<question>You run DBSCAN and all points are labeled as noise (-1). What is the most likely issue?

- <variant>eps is too large
- <variant>min_samples too small
- <variant>Too many features
- <variant>Scaling unnecessary
- <variantright>eps is too small

<question>You run DBSCAN and get only one large cluster. What is the most likely cause?

- <variant>eps is too small
- <variant>min_samples too large
- <variant>Too many features
- <variant>No scaling
- <variantright>eps is too large

<question>You increase min_samples significantly. What happens?

- <variant>More clusters form
- <variant>Clusters merge
- <variant>Distances shrink
- <variant>Model becomes faster
- <variantright>More points become noise

min_samples=5)db.fit(X_train)labels = db.fit_predict(X_test)

- <variant>DBSCAN cannot use test data
- <variant>fit_predict is invalid
- <variant>X_test must be labeled
- <variant>eps must be tuned
- <variantright>Second fit overwrites model

<question>You try to use DBSCAN like this:db.fit(X_train)labels = db.predict(X_test)What is the issue?

- <variant>DBSCAN cannot fit data
- <variant>predict needs y_test
- <variant>X_test must be scaled
- <variant>eps too small
- <variantright>DBSCAN has no predict

<question>You run DBSCAN and then compute silhouette score:labels = db.fit_predict(X_scaled)silhouette_score(X_scaled, labels)You get an error. Why?

- <variant>DBSCAN cannot cluster
- <variant>Scaling is incorrect
- <variant>Too many features
- <variant>eps too large
- <variantright>Noise label -1 included

<question>You try to tune DBSCAN like this:for eps in [0.1, 0.5, 1.0]: db = DBSCAN(eps=eps) labels = db.fit_predict(X_scaled) print(len(set(labels)))What is the flaw?

- <variant>eps cannot be looped
- <variant>fit_predict invalid
- <variant>X_scaled must be labels
- <variant>DBSCAN needs y
- <variantright>Noise counted as cluster

<question>How to fix cluster counting in DBSCAN?

<variant>Remove scaling

<variant>Use predict

<variant>Change eps

<variant>Reduce features

<variantright>Exclude label -1

<question>You run DBSCAN and get only noise. Then you scale data differently and clusters appear. Why?

<variant>Scaling creates clusters

<variant>DBSCAN ignores data

<variant>min_samples changes

<variant>eps removed

<variantright>Distance scale changed

<question>What does this code produce?`Z = linkage(X, method='ward')`

<variant>Cluster labels

<variant>Centroids

<variant>Distance matrix

<variant>Scaled data

<variantright>Hierarchy structure

<question>What does this code visualize?`dendrogram(Z)`

<variant>Cluster centers

<variant>Feature importance

<variant>Data distribution

<variant>Model accuracy

<variantright>Cluster hierarchy

<question>What does this code do?`fcluster(Z, t=5, criterion='distance')`

<variant>Fix number of clusters

<variant>Scale features

<variant>Remove noise

<variant>Split dataset

<variantright>Cut by distance

<question>What happens if distance threshold is very small?

<variant>Few clusters

<variant>No clusters

<variant>Data removed

<variant>Clusters merge

<variantright>Many clusters

<question>What does this code return?`Z.shape`

<variant>Cluster labels

<variant>Feature count

<variant>Sample count

<variant>Centroids

<variantright>Merge steps info

<question>You use `method='single'` in `linkage`. What happens?

<variant>Clusters become spherical

- <variant>Faster convergence
- <variant>No clusters
- <variant>Data removed
- <variantright>Chain-like clusters

<question>What is the issue in this code?
`pca = PCA(n_components=2)`
`X_pca = pca.fit_transform(X)`
`X_train, X_test = train_test_split(X_pca)`

- <variant>PCA cannot reduce dimensions
- <variant>`train_test_split` needs labels
- <variant>`n_components` must be 1
- <variant>`X` must be scaled after PCA
- <variantright>PCA applied before split

<question>What is the correct workflow to avoid leakage with PCA?

- <variant>Apply PCA to full dataset
- <variant>Apply PCA only to test
- <variant>Skip PCA in training
- <variant>Fit PCA after prediction
- <variantright>Fit PCA on train only

<question>What is the issue here?
`scaler = StandardScaler()`
`X_scaled = scaler.fit_transform(X)`
`pca = PCA(n_components=2)`
`pca.fit(X_scaled)`
`X_test_pca = pca.transform(X_test)`

- <variant>PCA cannot transform data
- <variant>Scaling is unnecessary
- <variant>`X_test` must be labeled
- <variant>Too many components
- <variantright>`X_test` not scaled

<question>What does this code return?
`pca.components_`

- <variant>Transformed data
- <variant>Cluster labels
- <variant>Feature values
- <variant>Distances
- <variantright>Principal directions

<question>What does this code return?
`pca.explained_variance_ratio_`

- <variant>Model accuracy
- <variant>Feature importance
- <variant>Cluster centers
- <variant>Distances
- <variantright>Variance explained

<question>You run PCA and get:
`explained_variance_ratio_ = [0.9, 0.05, 0.05]`
What does this mean?

- <variant>All components equal
- <variant>No dimensionality reduction
- <variant>Model overfits
- <variant>Features are random
- <variantright>First component dominates

<question>You set n_components=1 but model performance drops. Why?

<variant>PCA cannot reduce data

<variant>Scaling failed

<variant>Too many samples

<variant>Wrong metric used

<variantright>Information loss occurred

<question>What happens if you increase n_components?

<variant>Less variance kept

<variant>Fewer features

<variant>More compression

<variant>Faster training

<variantright>More variance preserved

<question>What is the issue here?
pca = PCA(n_components=0.95)
X_pca = pca.fit_transform(X_scaled)

<variant>n_components must be integer

<variant>PCA cannot use scaling

<variant>X_scaled invalid

<variant>Too many components

<variantright>No issue, valid usage

<question>What does n_components=0.95 mean?

<variant>Use 95 features

<variant>Reduce to 95 samples

<variant>Fix components to 95

<variant>Remove 5% data

<variantright>Keep 95% variance

<question>You use PCA for classification but accuracy drops. What is the likely reason?

<variant>PCA increases features

<variant>Scaling failed

<variant>Too many samples

<variant>Model incorrect

<variantright>Important info lost

<question>What is the key difference between LDA and PCA?

<variant>Both use labels

<variant>Both are unsupervised

<variant>Both maximize variance

<variant>Both reduce dimensions equally

<variantright>LDA uses class labels

<question>What is the main goal of LDA?

<variant>Maximize variance

<variant>Reduce noise

<variant>Scale features

<variant>Cluster data

<variantright>Separate classes

<question>What is a key difference between PCA and SVD?

<variant>PCA uses labels

<variant>SVD uses labels
<variant>PCA cannot reduce data
<variant>SVD cannot scale data
<variantright>PCA uses covariance

<question>What happens if you keep only top singular values?

<variant>Increase noise
<variant>Lose all data
<variant>No transformation
<variant>More features
<variantright>Reduce dimensionality

<question>What is the issue in this pipeline? `pipe = Pipeline([('svd', TruncatedSVD(n_components=2)), ('scaler', StandardScaler())])`

<variant>SVD cannot be used
<variant>Pipeline invalid
<variant>Scaling unnecessary
<variant>Too many components
<variantright>Wrong order

<question>What is the main constraint in NMF?

<variant>Matrix must be square
<variant>Values must be integers
<variant>Data must be normalized
<variant>Features must be independent
<variantright>All values must be non-negative

<question>What is the goal of NMF?

<variant>Predict labels
<variant>Cluster samples only
<variant>Reduce dataset size
<variant>Remove noise
<variantright>Factorize into parts

<question>What is the issue in this code? `model = NMF(n_components=2) X_new = model.fit_transform(X)` X contains negative values

<variant>NMF cannot reduce data
<variant>n_components too small
<variant>fit_transform invalid
<variant>X must be scaled
<variantright>Negative values not allowed

<question>What is the correct reconstruction?

<variant>`model.inverse_transform(X)`
<variant>`model.transform(W)`
<variant> $X = W + H$
<variant> $X = H \times W$
<variantright>`model.inverse_transform(W)`

<question>What happens if data is very sparse?

<variant>NMF fails
<variant>No components found

- <variant>Faster training
- <variant>Less variance
- <variant>NMF works well

<question>What is the issue in this pipeline?`pipe = Pipeline([('scaler', StandardScaler()),('nmf', NMF(n_components=2))])`

- <variant>NMF cannot be used
- <variant>Pipeline invalid
- <variant>Too many components
- <variant>Scaling unnecessary
- <variant>Scaling breaks non-negativity

<question>What is the correct approach for pipeline with NMF?

- <variant>StandardScaler → NMF
- <variant>NMF → StandardScaler
- <variant>Skip preprocessing
- <variant>Random scaling
- <variant>Use MinMaxScaler

<question> Optimal number of clusters in silhouette method is:

- <variant> Maximizing average silhouette score
- <variant> Minimizing distance
- <variant> Random selection
- <variant> Max clusters

<question> Time complexity of naive agglomerative clustering is:

- <variant> $O(n^3)$
- <variant> $O(n)$
- <variant> $O(\log n)$
- <variant> $O(n \log n)$

<question> What is PCA mainly used for?

- <variant> Dimensionality reduction
- <variant> Classification
- <variant> Clustering only
- <variant> Regression

<question> What does positive covariance indicate?

<variantright> Variables increase together

<variant> Variables decrease together

<variant> No relationship

<variant> Random behavior

<question> What is DBSCAN used for?

<variantright> Density-based clustering

<variant> Regression

<variant> Classification

<variant> Sorting

<question> What is dimensionality reduction?

<variantright> Reducing number of features

<variant> Increasing data size

<variant> Labeling data

<variant> Sorting data

<question> What is a core point in DBSCAN?

<variantright> Point with at least MinPts neighbors

<variant> Noise point

<variant> Centroid

<variant> Border only

<question> What is truncated SVD?

<variantright> Reduced feature matrix factorization

<variant> Classification model

<variant> Distance measure

<variant> Clustering algorithm

<question> What does NMF stand for?

<variantright> Nonnegative Matrix Factorization

<variant> Neural Matrix Function

<variant> Normalized Matrix Form

<variant> Nonlinear Matrix Factor

<question> What type of technique is NMF?

<variantright> Dimensionality reduction

<variant> Classification

<variant> Regression

<variant> Sorting

<question> Why may NMF lose more information than PCA?

<variantright> Due to non-negativity constraint

<variant> Due to labels

<variant> Due to clustering

<variant> Due to randomness

<question> What is the optimization method used in NMF?

<variantright> Alternating minimization

<variant> Gradient boosting

<variant> Random search

<variant> Backpropagation

<question> Which algorithm commonly uses bagging?

<variantright> Random Forest

<variant> K-Means

<variant> Linear Regression

<variant> PCA

<question> What is the main goal of ensemble methods?

<variantright> Improve prediction accuracy

<variant> Reduce dataset size

<variant> Increase training speed only

<variant> Remove features

<question> Boosting combines models in what way?

<variantright> Sequentially

<variant> Randomly

<variant> Independently

<variant> Simultaneously without order

<question> Which boosting algorithm is widely known?

<variantright> AdaBoost

<variant> Naive Bayes

<variant> KNN

<variant> SVM

<question> What is a key difference between bagging and boosting?

<variantright> Bagging trains models independently, boosting sequentially

<variant> Bagging reduces bias, boosting reduces variance

<variant> Bagging uses weights, boosting does not

<variant> No difference